Assignment 1: Shuttle Dispatch

Value and Policy Iteration

GitHub username: FailedMath3U **Repository:** MLDS
`https://github.com/FailedMath3U/MLDS`

1 MDP and modeling assumptions

At decision period t, the state $s_t \in \{0, \dots, 200\}$ is the number waiting before dispatch. The action $a_t \in \{0, 1\}$ means wait or dispatch one shuttle. Its capacity is 15. Let N_t be the subsequent arrivals, independently uniform on $\{1, 2, 3, 4, 5\}$. Define the passengers remaining after service as $q(s, a) = \max\{s - 15a, 0\}$. The cost and next state are

$$c(s, a) = 100a + 2q(s, a), \qquad s_{t+1} = \min\{q(s_t, a_t) + N_t, 200\}.$$

Thus dispatch precedes holding cost, and new arrivals enter afterward. Queue length is sufficient for the Markov property under independent arrivals and unrestricted shuttle availability.

The transition probability $P(s' \mid s, a)$ is the probability of next queue s' given state s and action a:

$$P(s' \mid s, a) = \frac{1}{5} \sum_{k=1}^{5} \mathbb{I}\{s' = \min\{q(s, a) + k, 200\}\}.$$

$\mathbb{I}\{\cdot\}$ equals one when its condition holds and zero otherwise. Arrival outcomes that reach the same capped state have their probabilities added.

Additional assumptions. At most one shuttle departs each period; there is no fleet-size or travel-time constraint. Overflow above 200 is discarded without penalty. New arrivals first incur holding cost at the next decision. The finite-horizon problem has 500 decisions ($t = 0, \dots, 499$) and no terminal cost. Ties choose waiting.

Objective and notation. With discount $\gamma = 0.95$, minimize expected discounted cost. $J_t(s)$ is the minimum remaining cost from state s at time t, discounting relative to that time:

$$J_t(s) = \min_{\pi} \mathbb{E}_{\pi}\left[\sum_{u=t}^{499} \gamma^{u-t} c(s_u, a_u) \,\middle|\, s_t = s\right].$$

Here u counts future periods, π is a rule for choosing actions, and $\mathbb{E}_{\pi}$ averages arrival randomness under that rule. For an infinite horizon, $J^*(s)$ denotes the optimal cost with no deadline. Plots show cost values: lower is better. If rewards are defined as $-c$, the corresponding reward values are $-J$.

Run the code. From `assignment1/mlds_assignment1_shuttle/`:

```
python -m pip install -r requirements.txt
python shuttle.py
jupyter lab shuttle_plots.ipynb
```

The solver runs all three methods and 14 validation experiments. Evidence is in `results/summary.json`; `state_results.csv` lists all states, and `shuttle_results.npz` saves every finite-horizon value/policy. The notebook regenerates plots. Section 5 explains each validation experiment.

2 Part (a): enumeration, $T = 500$

We enumerate every state, action and arrival outcome by backward induction. Set $J_{500}(s) = 0$; then for $t = 499, \ldots, 0$ compute

$$Q_t(s,a) = c(s,a) + \frac{\gamma}{5} \sum_{k=1}^{5} J_{t+1}(\min\{q(s,a) + k, 200\}), \qquad J_t(s) = \min_{a \in \{0,1\}} Q_t(s,a).$$

$Q_t(s,a)$ is the cost of action a now followed by optimal decisions. The minimizing action defines the time-dependent policy $\pi_t(s)$. All 501 value vectors and 500 policy vectors are saved. This reuses continuation values rather than listing exponentially many complete histories; an independently enumerated small tree validates the recursion (Section 5).

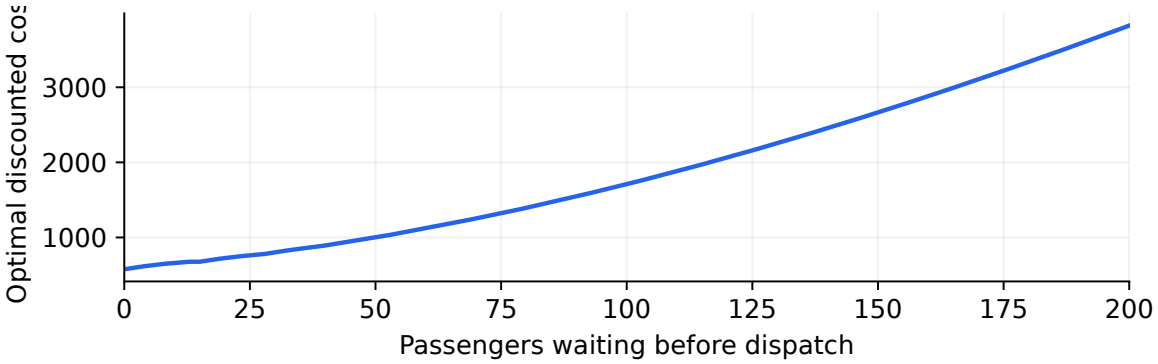


Figure 1: Part (a): optimal discounted cost at time zero, $J_0(s)$, for 500 decisions and zero terminal cost.

3 Part (b): infinite-horizon value iteration

Starting from $J^{(0)}(s) = 0$, update

$$J^{(n+1)}(s) = \min_{a \in \{0,1\}} \left[c(s,a) + \frac{\gamma}{5} \sum_{k=1}^{5} J^{(n)}(\min\{q(s,a) + k, 200\}) \right].$$

The index n counts iterations, not decision periods. Let $\mathcal{T}J$ denote this entire update applied to a candidate vector J. Stop when the Bellman residual $\delta(J) = \max_s |(\mathcal{T}J)(s) - J(s)|$ is at most 10^{-10}. This took **518 updates**, with final residual 9.913×10^{-11}.

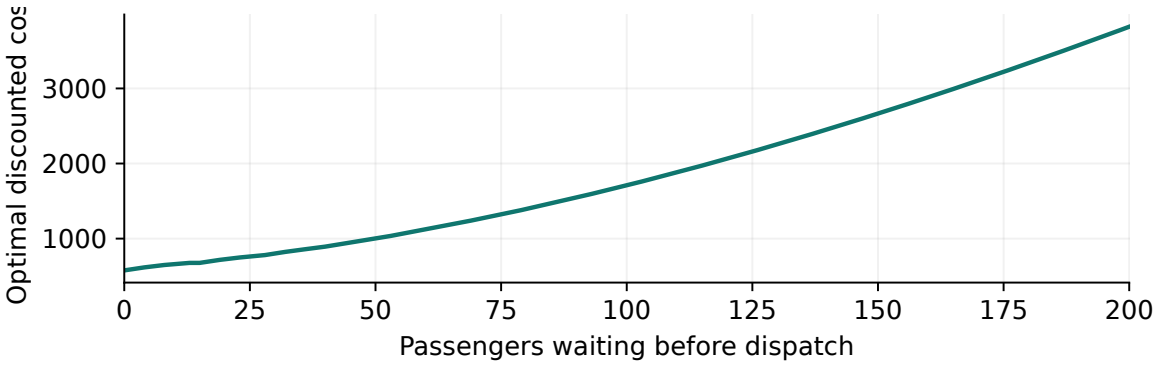


Figure 2: Part (b): infinite-horizon optimal cost $J^*(s)$ from value iteration. Its near agreement with Part (a) is tested using a discounted-tail bound.

4 Part (c): policy iteration and optimal policy

Start from always waiting. For a stationary policy π, form its transition matrix P_π and cost vector c_π, where $P_\pi(s, s') = P(s' \mid s, \pi(s))$ and $c_\pi(s) = c(s, \pi(s))$. Its value vector J_π satisfies

$$(I - \gamma P_\pi) J_\pi = c_\pi.$$

I is the identity matrix. Solve this linear system, then improve each state's action by comparing current cost plus discounted continuation under J_π. Repeat until no action changes. Policy iteration took **six evaluation/improvement rounds**, including the final stability check.

The optimal stationary policy is to **wait at queues 0–12 and dispatch at queues 13–200**. No threshold restriction was imposed on the search.

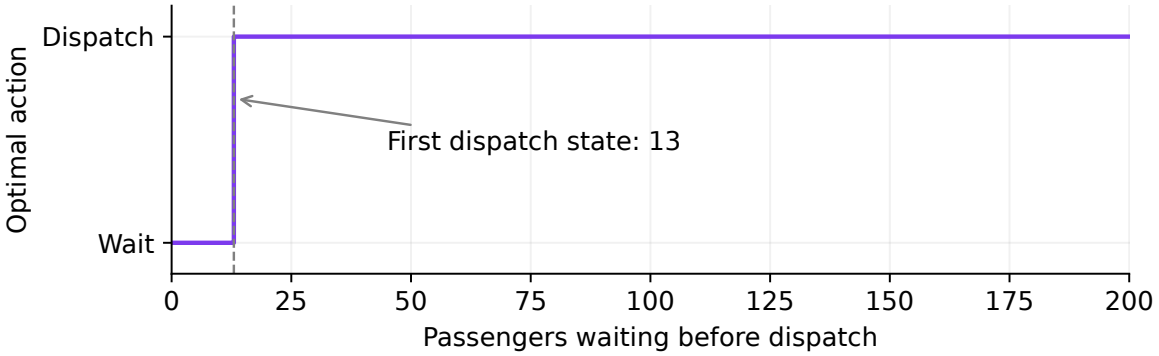


Figure 3: Part (c): optimal stationary policy. Dispatch begins at 13 passengers, before the shuttle is full.

Why dispatch before 15? The action costs, including optimal future costs, switch order between 12 and 13:

Queue	Wait cost	Dispatch cost	Decision
12	672.892	676.670	Wait
13	680.770	676.670	Dispatch

Waiting for a full shuttle is feasible but more expensive: exact policy evaluation gives cost 597.356 from an empty queue, versus 576.670 for the optimal policy. Representative optimal costs are $J^*(15) = 676.670168$ and $J^*(200) = 3821.586477$.

Edge cases and their consequences

Empty and underfilled shuttles. The maximum with zero prevents negative queues. Dispatching at 0 costs 100 extra without changing the next-state distribution, so it is dominated. Dispatch at 15 clears the queue; at 16 it leaves one. Also, $J^*(15) - J^*(0) = 100$, since optimal dispatch at 15 and waiting at 0 leave identical queues before arrivals.

Station capacity. At 200, waiting leads to 200 with probability one; dispatch leads to 186–190 with probability 1/5 each. Repeated capped outcomes must be pooled, not treated as distinct next states with lost probability.

The final decision. Waiting is optimal everywhere at $t = 499$: dispatch costs 100 but saves at most $2 \times 15 = 30$ immediately. With no terminal penalty, there is no later benefit. The saved final-period policy and terminal value were checked against this prediction; the infinite-horizon rule must not be used at every finite-horizon time.

5 Quantitative validation and execution

All **14 saved validation experiments passed**. Agreement alone is insufficient because solvers share a model; the tests also use analytic modifications, independent tree enumeration and independently coded simulation.

Optimality and solver agreement. Value and policy iteration choose identical actions in all 201 states; their maximum value difference is 1.964×10^{-9}. Policy iteration's Bellman residual is 9.095×10^{-13}. Since discounting makes the update contractive,

$$\max_s |J(s) - J^*(s)| \leq \frac{\delta(J)}{1-\gamma},$$

so the residual implies a value-error bound of 1.819×10^{-11}, subject to rounding. The smallest action-cost gap is 3.778, far larger than numerical error. Finite-horizon time-zero values differ from infinite-horizon values by at most 4.942×10^{-9}, below the omitted-tail bound $470\gamma^{500}/(1-\gamma) = 6.838 \times 10^{-8}$; 470 bounds any stage cost.

Modified problems with known answers. Each prediction below matched all states:

Modification	Predicted and observed result
Holding cost set to zero	Always wait; optimal cost zero.
Zero capacity and no arrivals	Always wait; $J(s) = 2s/(1-\gamma)$; zero analytic error.
Free dispatch, capacity 200, no arrivals	Dispatch at nonempty queues; cost zero.
Discount set to zero	Always wait; $J(s) = 2s$.

An exhaustive scalar decision/arrival tree with seven states and four periods matched backward induction exactly. This checks the recursion without reusing its vectorized transition arrays.

Independent simulation. For each initial queue, simulate 20,000 trajectories of 300 decisions with seed 1668, coding costs/transitions directly rather than using the solver's array builder:

Initial queue	Simulated mean	Exact policy cost	Standard error
0	576.671469	576.670168	0.219213
15	676.814669	676.670168	0.221520
200	3821.605190	3821.586477	0.532280

Standard error is trajectory-cost sample SD divided by $\sqrt{20{,}000}$. Every discrepancy is below one SE; the automated tolerance is four SE plus a 0.001951 omitted-tail bound. Simulation checks policy evaluation; Bellman optimality supplies the optimality evidence.

Rebuild the report. Run `python make_report_figures.py`, then compile `SHUTTLE_REPORT.tex`. The builder audits saved arrays and reproduces the full-shuttle comparison without rerunning or overwriting the original solvers.

With three additional months: test cost, capacity, arrival and queue-cap sensitivity; compare overflow penalties and terminal costs. Independently solve the MDP by linear programming and examine near-tied actions at higher precision. Finally, test the policy under correlated demand and explicit shuttle availability/travel times, adding those variables to the state where necessary. These distinguish numerical verification from robustness to modeling assumptions.

Assignment 1: Policy Gradient

CartPole-v1 and Pong-v5, Parts 1 and 2

GitHub username: FailedMath3U **Repository:** MLDS
`https://github.com/FailedMath3U/MLDS`

1 Algorithm and experimental protocol

The PyTorch implementation uses REINFORCE: collect complete episodes, compute discounted reward-to-go, then update the policy once per batch. The policy $\pi_\theta(a_t \mid o_t)$ gives action probabilities from network input o_t; θ denotes all actor weights and biases. At decision t, the sampled action is a_t and the subsequent reward is r_t. For episode length T and discount γ, optimize

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T-1} \gamma^t r_t\right], \qquad G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k.$$

G_t is the discounted reward remaining from decision t, calculated backward with zero continuation at the episode end. For B episodes, Adam minimizes

$$\mathcal{L} = -\frac{c}{B} \sum_{i=1}^{B} \sum_{t=0}^{T_i-1} \gamma^t \log \pi_\theta(a_t^{(i)} \mid o_t^{(i)})(G_t^{(i)} - b_t^{(i)}).$$

Here i indexes episodes, b_t is the baseline (zero in Part 1), and c is a fixed positive scale. The code sums decision contributions and divides by the number of episodes, not their total length. Returns and baseline predictions are held fixed during actor differentiation. The two output logits define a categorical action distribution. There is no return normalization or centering. Pong accumulates gradients in chunks but takes only one optimizer step per complete batch.

Actor setting	CartPole	Pong
Network	$4 \to 64$ tanh $\to 2$ logits	$6400 \to 200$ ReLU $\to 2$ logits
Discount γ; loss scale c	0.95; 0.05	0.99; 1
Adam learning rate	0.01	0.001
Episodes per update; total per seed	32; 8,000	8; 20,000
Training seeds	1668, 1669, 1670	1668
Gradient-norm cap	10	5
Hardware	CPU	One GPU

Evaluation protocol. Each required histogram uses 500 fresh episodes from the final fixed-budget policy, with no learning. Actions are sampled, not chosen greedily. Test reset seeds are separate from training and shared across methods. CartPole seed 1668 was fixed before the full runs for the main plots; all three seeds are reported. Every figure uses undiscounted episode reward $\sum_t r_t$; SD is the population standard deviation across its 500 evaluations.

Alternative Pong learning rate. On identical real-game data, one update at 0.001 produced mean action-distribution KL divergence 1.996×10^{-6}, versus 3.423×10^{-4} at 0.01. This motivated a smaller step; it does not prove greater long-run stability. Clipping affected 61.68% of basic Pong updates and 18.64% with a baseline. No recorded CartPole update reached its cap.

2 Part 1(a): CartPole-v1 without a baseline

Model. The four inputs are cart position, cart velocity, pole angle and pole angular velocity. Actions 0 and 1 apply left and right force. The default reward is +1 per transition, including failure. Physical failure occurs beyond cart position ± 2.4 or pole angle $\pm 12°$; a time limit stops episodes at 500 steps. Each reset independently samples the four components within $(-0.05, 0.05)$ using recorded episode seeds. We optimize this capped episode, so returns stop at either end flag without bootstrapping. The critic in Part 2 additionally observes time; the actor does not.

Result. The primary final policy scored **500.000 ± 0.000**. All 500 evaluations reached the limit without physical failure. An untrained network scored 20.926±11.467 and a uniform random policy 22.782±12.058 over 500 episodes each. Learning was substantial but not stable throughout training; Section 8 tests the large dips directly.

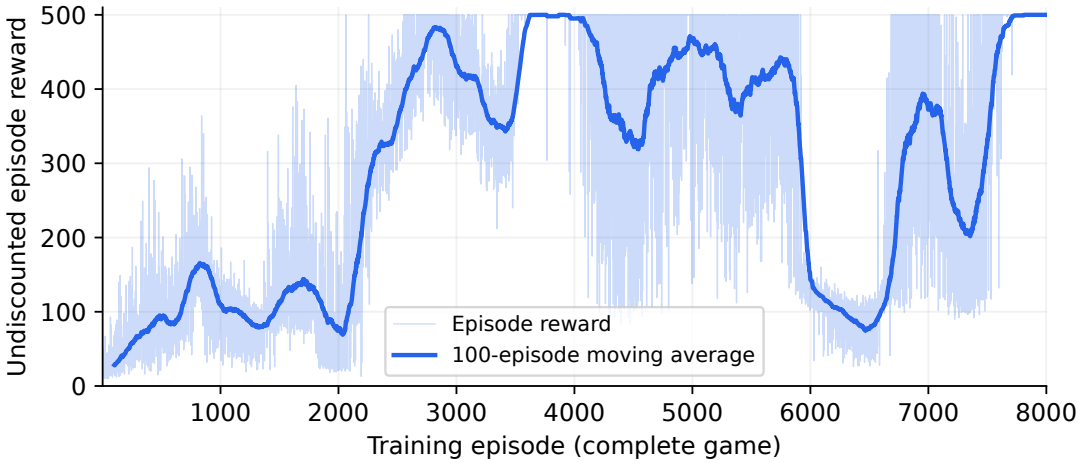


Figure 1: CartPole, Part 1: every training episode reward and trailing 100-episode mean. Seed 1668; 8,000 complete episodes.

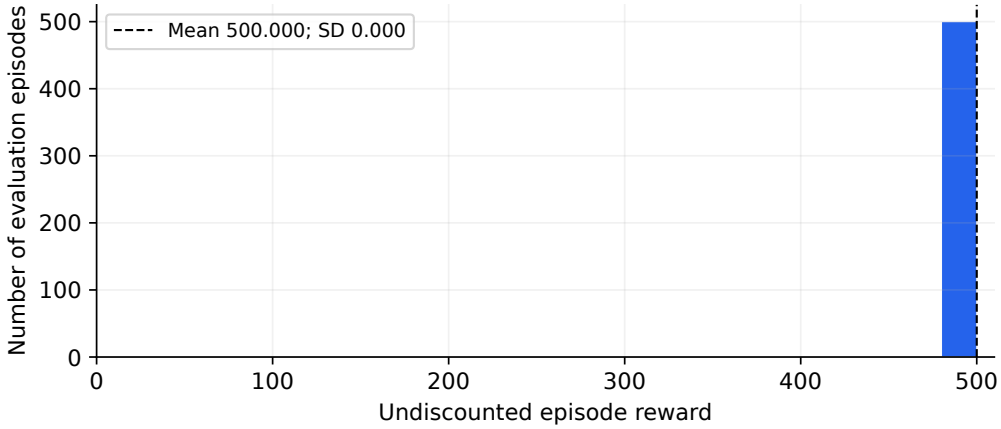


Figure 2: CartPole, Part 1: 500 final-policy evaluation rewards. Mean 500.000; population SD 0.000.

3 Part 1(b): Pong-v5 without a baseline

Model. We use `ALE/Pong-v5`, the namespaced Pong-v5 environment, with only actions 2/3 (ALE labels RIGHT/LEFT; vertical paddle movement). Rewards are +1 when the agent scores, −1 when the opponent scores, and zero otherwise. Episodes are complete games, not points. The assignment's crop, factor-two downsampling and background removal produce binary 80×80 frames. The network receives the flattened signed difference of consecutive frames; the first input after reset is zero.

This is a reactive policy in a partially observable process: o_t supplies motion cues but is not the full emulator state s_t. Unchanged image regions cancel, and the network has no recurrent memory. Transition probabilities are supplied implicitly by the emulator, not learned. Defaults retain four emulator frames per decision, sticky-action probability 0.25 and a 108,000-frame cap.

Result. The final policy scored **-18.610 ± 1.681**, versus an untrained reference of -20.250 ± 0.935 over 32 different games. All 500 test games were losses (range −21 to −10), with no truncations. **The policy did not learn to win.** The reference comparison is descriptive, not a paired test.

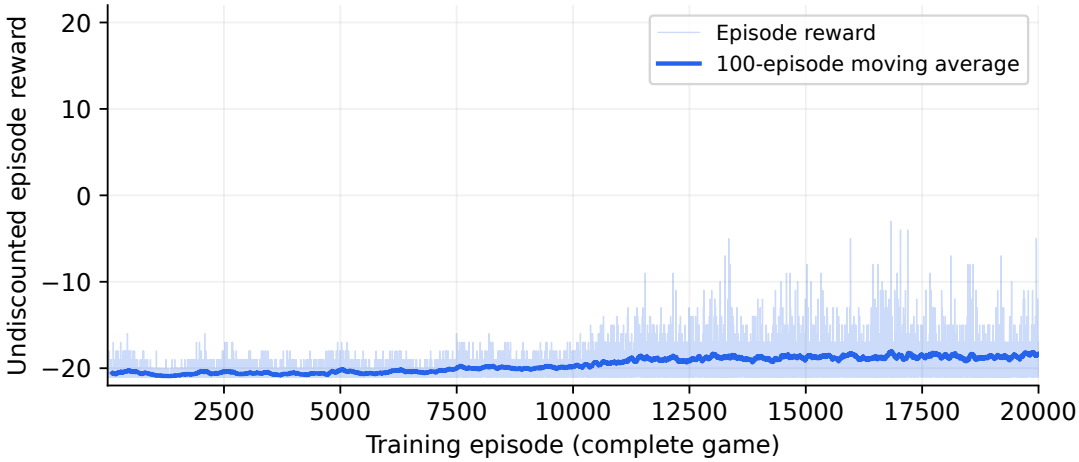


Figure 3: Pong, Part 1: every training episode reward and trailing 100-episode mean. Seed 1668; 20,000 full games.



Figure 4: Pong, Part 1: 500 final-policy evaluation rewards. Mean -18.610; population SD 1.681.

4 Part 2(a): CartPole with a learned baseline

Choice and implementation. A separate network with inputs $(o_t, t/500)$, 64 tanh units and one output predicts G_t. Time matters because fewer rewards remain near the episode limit. Before each actor update, its predictions are computed without gradients. Afterward, five Adam steps at rate 0.01 fit the observed Monte Carlo returns by mean squared error. There is no temporal-difference bootstrap. Actor settings, initial actor weights and episode budgets match Part 1.

Subtracting an action-independent baseline preserves the expected *unclipped* policy gradient:

$$\mathbb{E}_{a\sim\pi_\theta}[b\nabla_\theta \log \pi_\theta(a \mid o)] = b\nabla_\theta \sum_a \pi_\theta(a \mid o) = 0.$$

Here ∇_θ is the vector of derivatives with respect to actor parameters. This identity does not guarantee lower variance for an inaccurate baseline.

Result. The final model scored **260.308 ± 57.013**: four episodes reached 500, and 496 ended in physical failure. It improved over random control but underperformed the primary basic policy. The variance experiment in Section 6 distinguishes a baseline's local variance benefit from successful training.

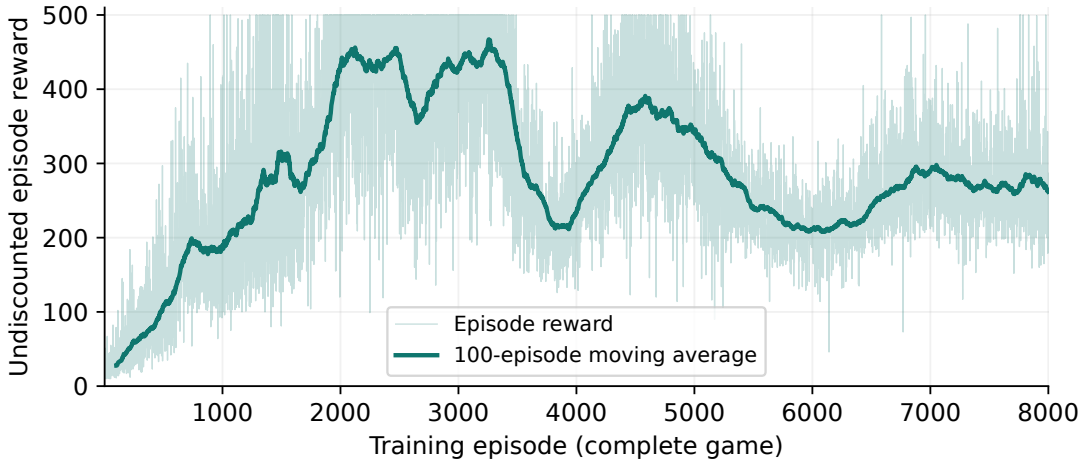


Figure 5: CartPole, Part 2: every training episode reward and trailing 100-episode mean. Seed 1668; 8,000 complete episodes.

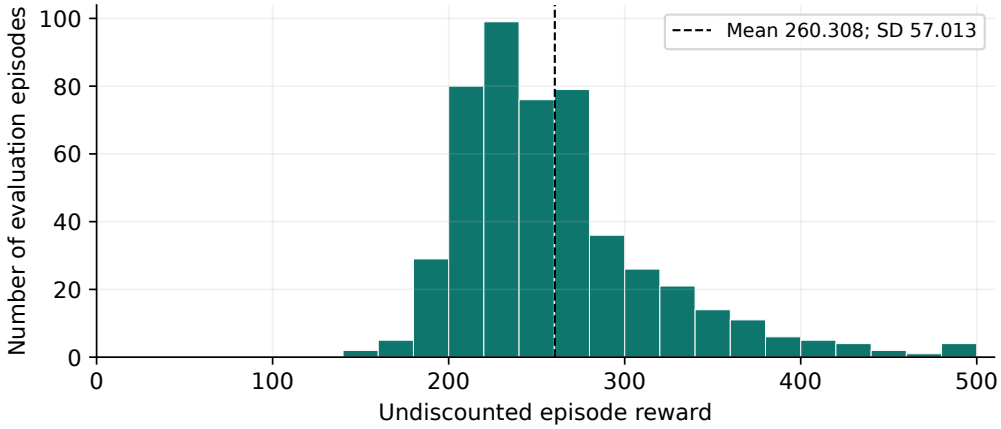


Figure 6: CartPole, Part 2: 500 final-policy evaluation rewards. Mean 260.308; population SD 57.013.

5 Part 2(b): Pong with a constant baseline

Choice and implementation. The baseline is a scalar exponential moving average (EMA), held constant within each batch. Starting at $b_0 = 0$, update k uses only the previous value; afterward,

$$\bar{G}_k = \frac{\sum_{i,t} \gamma^t G_t^{(i)}}{\sum_{i,t} \gamma^t}, \qquad b_{k+1} = 0.95 b_k + 0.05 \bar{G}_k.$$

k counts actor updates. Thus the current sampled actions do not determine the baseline used for their own gradient. Apart from this subtraction, the actor architecture, initial weights, environment, budgets and action restrictions match Part 1.

Result. The final model scored **-18.446 ± 1.851**, 0.164 above the basic method. All 500 games were losses (range −21 to −5), with no truncations. One training seed does not establish a general benefit. Later paired probes found no consistent variance reduction from the actual EMA (Section 7). Frequent gradient clipping also means actual updates differ from the unclipped estimator used in the baseline identity.

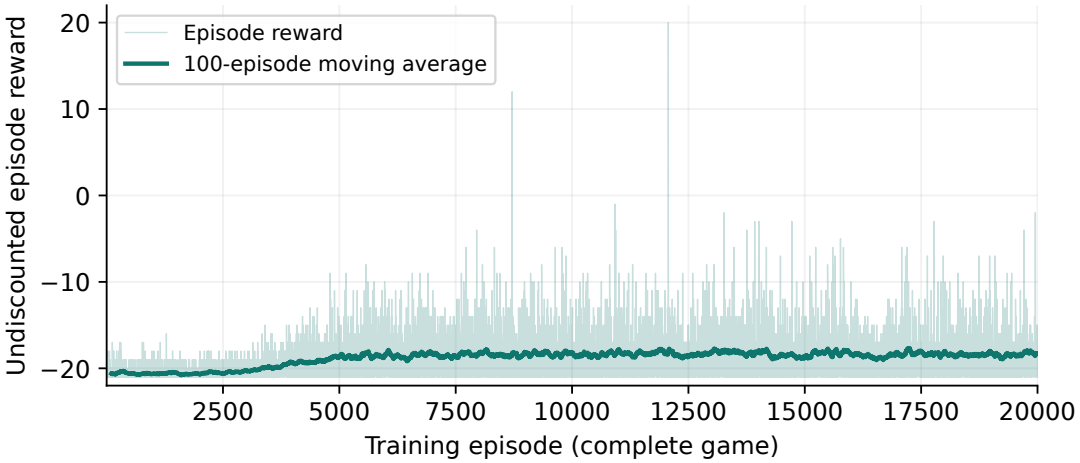


Figure 7: Pong, Part 2: every training episode reward and trailing 100-episode mean. Seed 1668; 20,000 full games.



Figure 8: Pong, Part 2: 500 final-policy evaluation rewards. Mean -18.446; population SD 1.851.

6 CartPole: edge cases and validation

Boundary crossings and unavoidable failure. Supplementary checks tested both actions at both signed cart/pole limits, and just inside each limit with outward velocity: all 16 predictions matched. Equality at the threshold is allowed; crossing it terminates. For example, $x = 2.39$ and $\dot{x} = 1$ give $x' = 2.41$ under either action because the default Euler step uses $x' = x + 0.02\dot{x}$. Force changes velocity too late to avoid that transition's failure. The terminal reward remained +1.

Failure and time limit together. Setting a one-step limit and forcing a boundary crossing returned both `terminated=True` and `truncated=True`, with one reward of 1. A score of 500 alone therefore cannot identify failure-free survival; both flags are retained. The separate original time-limit check verified truncation without failure.

Different episode lengths. The actual collector was exercised with controlled episodes of lengths 1, 1 and 3. It recorded no post-end transitions; the one-step return was exactly 1. The combined loss gradient agreed with the mean of separately computed episode gradients to a maximum component error of 1.863×10^{-9}. This checks trajectory separation and averaging by episode rather than introducing length-dependent normalization.

Negative controls and recovery limits. Frozen primary actors and fixed-action controls were evaluated on 64 matched starts per condition, without retraining. Trained actors used *greedy* actions here, separately from the required stochastic evaluations:

Policy	Default resets	Tilted, moving outward	Near track edge
Always left	9.34	9.50	7.75
Always right	9.38	9.50	7.78
Basic, greedy	469.77	488.44	8.70
Baseline, greedy	283.61	284.77	13.33

Entries are mean episode rewards. Tilted starts set pole angle to ± 0.15 radians and angular velocity to ± 0.5; edge starts set cart position to ± 2.3 and velocity to ± 0.5. Signs alternate; other components retain seeded reset values. Fixed-direction controllers fail quickly, while trained policies balance from default and tilted starts. Both struggle near the track edge. These exploratory stress tests expose limits of standard-reset results; they do not establish that every edge start is recoverable. The basic actor's greedy default score also shows why evaluation action selection must be specified.

Baseline variance. At a frozen initial actor, a separate critic fitted on 512 trajectories was measured on 512 different, paired trajectories. Summed sample variances of gradient components fell from 5.338262 to 0.842421 (84.22%). The mean-gradient difference was 0.0474 versus paired standard-error magnitude 0.1176. This supports variance reduction at that actor, but the critic received 150 fitting steps; it does not establish the same benefit for five-step online fitting throughout training.

Decision inspection. The saved ten-step trace contains all four observations, probabilities, sampled actions and rewards. At step 3, right had probability 0.7711 but left was sampled; the pole subsequently remained within bounds. This is compatible with stochastic action selection, not evidence of a reversed action mapping. A separate fixed-seed, ten-action replay reproduced identical state sequences.

7 Pong and shared gradient validation

Scoring is not an episode end. For rewards $(0, 1, 0, -1)$ at $\gamma = 0.99$, independently calculated returns are

$$(G_0, G_1, G_2, G_3) = (0.019701, 0.0199, -0.99, -1).$$

The implementation matched within 10^{-6}. In particular, $G_1 = 1 - 0.99^2$ includes the later loss; incorrectly restarting returns at the positive point would give 1. Complete-game collection was checked separately. Both end flags stop collection, and frame history is reset between games.

Serving under restricted actions. Eight complete preflight games using only actions 2/3 each produced 21–22 scoring events and reached natural termination. No startup FIRE/NOOP wrapper was added. This verifies progress under the assigned restriction, not that FIRE has no effect in unrestricted play.

Sticky actions. The emulator can repeat its previously executed action with probability 0.25 per emulator frame. Requested and executed actions therefore need not coincide throughout a four-frame decision. We retain this stochasticity and test seeded replay, rather than assuming deterministic action execution: identical reset seeds and requested-action sequences reproduced frame/reward hashes. This establishes reproducibility, not robustness across all seeds.

Visual input. Known-pixel images verified the crop, factor-two sampling, background removal and foreground preservation. Subtraction uses signed values, so disappearing pixels become -1 rather than unsigned overflow. Real differences were in $\{-1, 0, 1\}$; reset input was zero. The saved game trace includes the first ten decisions, a point and the ending, with corresponding image differences. Its positive point at decision 93 and final loss at 1,797 illustrate how rewards can be far apart within one episode.

Does the Pong baseline reduce variance? An independently fitted candidate constant reduced summed gradient variance from 369.320 to 20.763 on 16 held-out games (94.38%). It was fitted on eight other games and was *not* the initial training EMA, which starts at zero. Later probes used the actual frozen EMA and 16 paired games at the same actor. Variance ratios (with/without baseline) at 4,096, 8,192, 12,288 and 16,384 games were 0.873, 1.017, 0.924 and 1.011. Reduction was not sustained. Small probe samples and the absence of ratio confidence intervals limit precision; all probes measure unclipped gradients.

Analytic gradient controls for both parts. In a two-step task with Bernoulli action probability $p = \text{sigmoid}(\theta)$ and reward equal to the action, the exact objective is $(1 + \gamma)p$. Enumeration over all four action sequences reproduced its derivative $(1 + \gamma)p(1 - p)$, with and without a constant baseline: error 8.52×10^{-9} for CartPole's discount and below 10^{-7} for Pong's. Setting discount to zero recovered immediate rewards. Zero Pong returns produced zero basic-policy gradient. A one-step actor check with target 1 changed the selected action's probability from 0.5 to 0.505000 with baseline 0, and to 0.495000 with baseline 2. This verifies both signs of the advantage, rather than assuming positive game rewards require every action probability to increase.

These controls test the estimator and task handling. They do not establish that the Pong representation, optimizer settings or training budget were adequate to learn winning play.

8 Tuning and interpretation

All CartPole seeds. Each row is a final policy evaluated on 500 stochastic episodes at the original actor rate 0.01:

Method	Seed	Mean	Population SD	Fraction at 500
No baseline	1668	500.000	0.000	1.000
No baseline	1669	271.824	28.856	0.000
No baseline	1670	483.886	65.298	0.928
Learned baseline	1668	260.308	57.013	0.008
Learned baseline	1669	360.516	79.141	0.100
Learned baseline	1670	349.940	76.321	0.092

The baseline improves seed 1669, but not the other two. The following additional runs changed only actor rate, keeping seeds, 8,000-episode budgets and 500 evaluations per model; critic rate remained 0.01:

Actor rate	Basic: mean across three seeds	Baseline: mean across three seeds
0.01	418.570	323.588
0.005	163.178	357.661
0.002	247.102	264.481

Smaller rates did not consistently improve final reward. These exploratory comparisons do not identify an optimal rate or establish that a baseline generally helps.

Are the training dips just episode noise? Both seed-1668 runs were replayed exactly: every original training reward and final parameter hash matched. Before/during checkpoints selected from those curves were then frozen and evaluated on 500 fresh paired reset/action seeds:

Method	Before mean	During mean	Change [paired 95% interval]
Basic (5,120 → 6,400)	453.108	83.114	−369.994 [−377.430, −362.038]
Baseline (3,072 → 6,144)	440.466	215.686	−224.780 [−231.886, −217.436]

Intervals use 5,000 paired bootstrap replicates. A further 2,000 episodes with each earlier actor frozen gave 100-episode mean ranges of 430.11–472.29 (basic) and 413.00–453.82 (baseline); continued training reached 74.64 and 211.37. Thus the selected dips reflect policy changes beyond measured fixed-policy episode noise. Noisy updates may still cause those changes. Selection was exploratory; intervals condition on these actors and do not measure uncertainty across training seeds. Rolling windows overlap, so their ranges are descriptive.

Objective versus displayed score. For CartPole, discounted return is $(1 - 0.95^T)/0.05$, close to 20 long before 500 steps. The primary baseline's mean discounted return was 19.999743 despite its lower survival time. In Pong, mean discounted return improved from −1.309180 for the untrained reference to 0.313913 (basic) and 0.293346 (baseline), while all final games were losses. A reward 1,000 decisions away has weight $0.99^{1000} \approx 4.32 \times 10^{-5}$. This suggests a credit-assignment difficulty; it does not identify the cause of failure. Partial observation, optimization and the single training seed remain unresolved limitations.

9 Execution and evidence

Paths are relative to the repository's `assignment1/`. Retrain in a fresh copy to preserve saved runs. Versions: Python 3.12, PyTorch 2.14.0, NumPy 2.5.3, Gymnasium 1.2.2; Pong also uses ALE 0.11.2.

CartPole:

```
cd mlds_assignment1_cartpole
python -m pip install -r requirements.txt
python cartpole.py suite --episodes 8000 --lr 0.01
python validate.py
python validate_task_edges.py
python instability_experiment.py
python plot_instability_experiment.py
```

The suite trains/evaluates six actors. `validate.py` writes analytic-gradient, return and baseline-variance evidence to `validation/`. `validate_task_edges.py` writes boundary, collector, advantage-sign and recovery evidence to `task_validation/`. The instability scripts reproduce replay, frozen-policy comparisons and bootstrap results. Each output folder contains numerical records; the original notebook plots saved runs. Lower-rate folders `cartpole_lr005/` and `cartpole_lr002/` provide `run.sh` for their complete suites.

Pong: install its requirements and copy source/config/supervisor into a fresh directory. Set one GPU UUID in `config.json` and `supervise.sh`, and configure the supervisor's Python executable. Activate that environment:

```
export CUDA_VISIBLE_DEVICES=<single-GPU-UUID>
export CUBLAS_WORKSPACE_CONFIG=:4096:8
export OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1
python run_pong.py preflight
nohup setsid bash supervise.sh > nohup.log 2>&1 < /dev/null &
```

Preflight runs the 15 checks; training alternates methods on one GPU and probes variance every 4,096 games. Existing checkpoints resume. Evidence: `preflight/results.json`, `runs/*/training.csv`, `runs/*/evaluation/test_final.json` and `diagnostics/`. The README explains recovery and traces. The original suite took 2.64 hours; CartPole used CPU only.

Report: run `python build_assets.py`, then compile this LaTeX source. The builder audits numerical results and regenerates figures. `python combine_submission.py` adds the shuttle PDF.

10 Experiments with three additional months

CartPole: use more training seeds and map recovery across initial positions, angles and velocities. Compare actor rate, batch size and critic-fitting frequency under matched budgets. Measure critic error and paired gradient variance throughout training to test whether critic lag explains instability.

Pong: keep v5, actions 2/3 and $\gamma = 0.99$ while comparing frame differences, frame stacks and a convolutional policy. Measure gradient contributions by decision time. Tune rate, clipping and batch size on validation seeds; evaluate multiple training seeds on untouched 500-game tests.

Baselines: compare scalar, state-dependent and time-dependent predictions at identical frozen actors with independent fitting data. Use hundreds of paired games, variance-ratio confidence intervals, and separate clipped/unclipped measurements to test whether variance reduction improves control.